

INTRO TO SCIENTIFIC COMPUTING IN PYTHON

LECTURE 1

Zan Ahmad

Department of Applied Mathematics and Statistics,
Johns Hopkins University

January 8, 2026

Part I

THINKING IN PYTHON

RECAP

So far, you have learned:

- ▶ basic Python syntax,
- ▶ core data structures,
- ▶ how to write functions and loops.

Before introducing NumPy, we need to learn how to:

- ▶ reason about Python code,
- ▶ choose the right data structure,
- ▶ recognize common algorithmic patterns.

Goal: learn how to *think* in Python, not just write code in it.

VARIABLES ARE NAMES, NOT BOXES

In Python, variables do not *store* values.

They are **names** that refer to objects.

Assignment means:

- ▶ bind a name to an existing object,
- ▶ not “copy the value”.

This distinction explains many surprising bugs.

EXAMPLE: SHARED REFERENCES

Consider the following code:

```
a = [1, 2, 3]
b = a
b.append(4)
```

Questions:

- ▶ What is a now?
- ▶ Why did it change?

Key idea: both a and b refer to the same list object.

MUTABILITY

Some Python objects can be modified in-place.

Mutable:

- ▶ lists
- ▶ dictionaries
- ▶ sets

Immutable:

- ▶ integers
- ▶ floats
- ▶ strings
- ▶ tuples

Why this matters:

- ▶ function behavior,
- ▶ debugging,
- ▶ scientific correctness.

FUNCTIONS CREATE LOCAL SCOPE

When a function is called:

- ▶ a new local scope is created,
- ▶ parameter names are bound to objects,
- ▶ execution happens inside that scope.

Understanding this prevents subtle bugs.

EXAMPLE: MUTATING INPUTS

```
def f(x):  
    x.append(10)
```

```
a = [1, 2]  
f(a)
```

Questions:

- ▶ Why does a change?
- ▶ Is this always bad?

Key idea: functions can mutate objects passed into them.

DESIGN GUIDELINE

Good scientific code:

- ▶ minimizes unexpected mutation,
- ▶ documents when mutation occurs,
- ▶ uses return values for clarity.

This becomes critical when working with large arrays later.

WHY DATA STRUCTURE CHOICE MATTERS

The same problem can be solved many ways.

Choosing the right data structure affects:

- ▶ clarity,
- ▶ correctness,
- ▶ performance.

Before writing code, ask:

- ▶ What information do I need to store?
- ▶ How will I access it?

LISTS

Lists are ordered sequences.

Use lists when:

- ▶ order matters,
- ▶ you need indexing,
- ▶ data grows over time.

Examples:

- ▶ time series,
- ▶ simulation outputs,
- ▶ collected measurements.

DICTIONARIES

Dictionaries map keys to values.

Use dictionaries when:

- ▶ you need fast lookup,
- ▶ data is naturally labeled,
- ▶ you are counting or grouping.

Examples:

- ▶ frequency counts,
- ▶ grouping observations,
- ▶ configuration parameters.

SETS

Sets store unique elements.

Use sets when:

- ▶ you care about uniqueness,
- ▶ membership testing is important.

Sets are often used for:

- ▶ detecting duplicates,
- ▶ tracking “seen” elements.

COMMON ALGORITHMIC PATTERNS

Many problems reduce to a small set of patterns.

Learning these patterns lets you:

- ▶ solve new problems faster,
- ▶ recognize structure,
- ▶ write clearer code.

PATTERN 1: ACCUMULATION

Compute a running total or aggregate.

Examples:

- ▶ sums,
- ▶ products,
- ▶ averages.

This pattern appears everywhere in scientific computing.

PATTERN 2: TRANSFORMATION

Apply the same operation to each element.

Examples:

- ▶ squaring values,
- ▶ unit conversion,
- ▶ normalization.

This will later become vectorized operations in NumPy.

PATTERN 3: FILTERING

Select elements that satisfy a condition.

Examples:

- ▶ removing outliers,
- ▶ thresholding data,
- ▶ conditional selection.

PATTERN 4: COUNTING AND FREQUENCY

Track how often things occur.

Examples:

- ▶ histograms,
- ▶ word counts,
- ▶ categorical statistics.

Dictionaries are the natural tool here.

PATTERN 5: TWO-PASS LOGIC

First pass: collect information.

Second pass: use that information.

Examples:

- ▶ find max, then normalize,
- ▶ compute statistics, then threshold,
- ▶ build index, then query.

WHY THIS MATTERS FOR NUMPY

NumPy does not introduce new ideas.

It:

- ▶ optimizes these same patterns,
- ▶ applies them to large arrays efficiently,
- ▶ moves computation closer to compiled code.

If you understand these patterns in Python, NumPy will feel natural.

Part II

BASICS OF PROGRAMMING: ALGORITHMIC THINKING

MOTIVATION

So far, we have focused on:

- ▶ writing correct Python code,
- ▶ understanding variables, loops, functions, and data structures.

However, correctness is only the *first step*.

New question:

- ▶ How does your code behave as the problem gets larger?

Key idea: Code that works for small inputs may fail completely at scale.

WHAT BIG-O MEASURES

Big-O notation describes how an algorithm scales as input size grows.

There are two main resources we care about:

- ▶ **Time complexity** — how runtime grows
- ▶ **Space complexity** — how memory usage grows

Big-O does *not* measure:

- ▶ exact runtime,
- ▶ wall-clock seconds,
- ▶ performance on a specific computer.

Big-O is about growth, not speed.

WHAT IS THE INPUT SIZE n ?

In Big-O analysis, n represents the size of the input.

Examples:

- ▶ length of a list,
- ▶ number of dictionary keys,
- ▶ size of a dataset,
- ▶ number of elements processed.

Big-O asks:

- ▶ What happens when n becomes very large?

Key idea: Scaling behavior matters more than small constant differences.

CONSTANT TIME: $O(1)$

An operation is $O(1)$ if it takes the same time regardless of input size.

Examples in Python:

- ▶ indexing into a list: `a[i]`
- ▶ dictionary lookup: `d[key]`

Whether the data has 10 elements or 10 million, the time is the same.

Key idea: $O(1)$ operations do not scale with data size.

LINEAR TIME: $O(n)$

An operation is $O(n)$ if it processes each element once.

Examples:

- ▶ looping through a list,
- ▶ `sum(a)`, `max(a)`, `len(a)`,
- ▶ searching for an element in a list.

If the input size doubles, the runtime doubles.

Key idea: Linear time means touching every element once.

QUADRATIC TIME: $O(n^2)$

Quadratic time typically arises from nested loops.

Example pattern:

- ▶ comparing every element with every other element,
- ▶ double loops over the same list.

If the input size doubles:

- ▶ the work increases by a factor of four.

Key idea: $O(n^2)$ algorithms become unusable very quickly.

LOGARITHMIC TIME: $O(\log n)$ AND $O(n \log n)$

Logarithmic time means the problem size is reduced by a constant factor at each step.

Typical behavior:

- ▶ the input is repeatedly halved,
- ▶ each step eliminates a large portion of the search space.

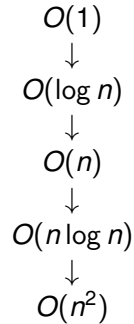
Classic example:

- ▶ binary search on a sorted list.

Important: Logarithmic time is *much faster* than linear time.

WHERE $\log n$ FITS IN BIG-O

Common time complexities ordered from fastest to slowest:



Key takeaways:

- ▶ $O(\log n)$ is closer to constant time than to linear time,
- ▶ $O(n \log n)$ is slower than linear but far better than quadratic,
- ▶ algorithm choice matters more than small constant factors.

SPACE COMPLEXITY

Space complexity measures how memory usage grows with input size.

Examples:

- ▶ Using a few variables: $O(1)$ space
- ▶ Creating a new list proportional to input: $O(n)$ space

Example:

- ▶ accumulating a sum uses constant space,
- ▶ building a transformed copy of a list uses linear space.

Key idea: Memory usage matters just as much as runtime.

LISTS: PERFORMANCE CHARACTERISTICS

Lists are ordered collections.

Common operations:

- ▶ index access: $O(1)$
- ▶ append: $O(1)$ (amortized)
- ▶ membership test x in a : $O(n)$
- ▶ insert or delete in middle: $O(n)$
- ▶ `sort()` and `sorted()` are $O(n \log(n))$

Key idea: Lists are fast for indexing, slow for searching.

DICTIONARIES AND SETS

Dictionaries and sets are hash-based data structures.

Typical operations:

- ▶ lookup: $O(1)$ average
- ▶ insertion: $O(1)$ average
- ▶ membership test: $O(1)$ average

This is why dictionaries and sets are extremely powerful.

Key idea: Dictionaries trade memory for speed.

SAME PROBLEM, DIFFERENT COMPLEXITY

Two correct solutions can behave very differently.

Example:

- ▶ checking for duplicates using nested loops: $O(n^2)$
- ▶ checking using a set: $O(n)$

Same language. Same problem. Orders of magnitude difference.

Key idea: Data structure choice determines performance.

WHY THIS MATTERS FOR SCIENTIFIC COMPUTING

Scientific datasets are often large.

Inefficient algorithms:

- ▶ waste compute resources,
- ▶ slow down experiments,
- ▶ limit what problems are tractable.

NumPy exists because:

- ▶ Python loops are slow at scale,
- ▶ vectorized operations reduce overhead.

Key idea: Algorithmic thinking enables scientific scale.

BIG-O AS A PROBLEM-SOLVING TOOL

For each problem, ask:

1. Does my solution work?
2. How does it scale?

In interviews and real projects:

- ▶ reasoning matters more than syntax,
- ▶ clarity matters more than cleverness.

Big-O is a way of thinking, not a formula to memorize.

NEXT: APPLYING THESE IDEAS

Now we will apply Big-O thinking to concrete problems.

We will:

- ▶ start with a naive solution,
- ▶ analyze its complexity,
- ▶ improve it using better data structures.

Goal: learn how algorithmic insight transforms solutions.

Part III

PRACTICE PROBLEMS

PRACTICE

Tomorrow's problems will use:

- ▶ lists,
- ▶ dictionaries,
- ▶ loops and logic,
- ▶ Big O notation.

Focus: reasoning, not syntax.

Your HW is to make sure you have a good grasp of all of these concepts. I have linked a nice YouTube video that covers all the syntax you need to know for coding interview questions.

WHAT'S NEXT?

Next lecture:

- ▶ Go over solutions to some practice problems
- ▶ NumPy arrays,
- ▶ vectorized computation,
- ▶ performance and broadcasting.